

Stoq

Smart Inventory & Grocery Manager

Product Requirements Document • v1.0

React Native • .NET Core 10 • PostgreSQL • Microsoft Identity

Generated: April 8, 2026

1. Product Overview

1.1 Product Vision

Stoq is a mobile-first application that helps families eliminate food waste, automate grocery list creation, and maintain a clear view of their home inventory. Built for households of 3 or more people, it prioritizes low-friction data entry through barcode scanning, receipt scanning, and AI-powered product recognition.

1.2 Problem Statement

Pain Point	Description
Food Waste	Families lose money buying items already at home or forgetting expiry dates.
Manual Effort	Maintaining grocery lists is repetitive and often abandoned within weeks.
Coordination	Multiple household members buy duplicates or miss needed items.
Inventory Blindness	No quick way to know what's available without physically checking.

1.3 Goals & Success Metrics

Metric	Target
Daily Active Use	≥ 3 sessions/week per household after 30 days
List Accuracy	≥ 85% of shopping trips covered by auto-generated list
Retention (30-day)	≥ 60% of households still active after 1 month
Expiry Awareness	Reduce reported food waste by ≥ 30% after 60 days

2. Users & Personas

2.1 Target Audience

Households of 3 or more people, primarily families with children or shared living arrangements. Primary users aged 25–50, moderately tech-savvy, using Android or iOS.

2.2 User Roles

Role	Capabilities
Admin (Owner)	Creates the household, manages members, configures product catalog and thresholds.
Member	Can view inventory, add/consume products, and edit the shopping list.
Viewer (optional)	Read-only access for shared list visibility (e.g., grandparent).

2.3 Key User Journeys

Journey A — First-time Setup

1. Download app and sign up (Google, Facebook, Apple, or email).
2. Create or join a household with a 6-digit invite code.
3. Add first products via barcode scan, photo, or manual entry.

Journey B — Weekly Shop

4. Open app to review auto-generated shopping list.
5. Edit list if needed (add, remove, check off items).
6. Return home and scan or photograph receipt to bulk-update inventory.

Journey C — Daily Consumption

7. Open product from inventory after using it.
8. Tap 'Use' to decrement quantity (or mark as finished).
9. System auto-adds to shopping list when quantity hits minimum threshold.

3. Core Features

3.1 Feature Overview

ID	Feature	Description
F-01	Authentication & Households	Multi-provider login; household creation with invite codes; role management.
F-02	Product Catalog	Barcode scan + manual entry; AI photo recognition; per-product minimum quantity and expiry rules.
F-03	Inventory Management	Add/remove/consume products; quantity tracking; expiry date tracking per batch.
F-04	Shopping List	Auto-generated from thresholds; real-time sync across household; manual additions; check-off during shopping.
F-05	Expiry Alerts	Push notifications at 3 days and 1 day before expiry; in-app expiry dashboard.
F-06	Receipt Scanning	Photo or NF-e QR Code scan; AI parses items; user confirms and batch-registers to inventory.
F-07	Notifications	Expiry alerts; low stock alerts; list updates from other household members.
F-08	Household Sync	All data synced in real-time via SignalR; offline queue for operations while disconnected.

3.2 Feature Details

F-01 — Authentication & Households

- Sign-in providers: Google, Facebook, Apple ID, and Email/Password.
- Microsoft Identity (ASP.NET Core Identity) as the identity backbone with JWT tokens.
- OAuth2 / OIDC flows for social providers via external login scheme.
- Households identified by UUID; invite via 6-digit alphanumeric code (expires in 48h).
- Admins can remove members, change roles, and regenerate invite codes.

F-02 — Product Catalog

- Barcode scan using device camera (ZXing or ML Kit); queries Open Food Facts API as primary source.
- AI photo recognition via Claude Vision API — identifies product name, brand, and category from photo.
- Manual fallback: name, brand, category, unit (g, ml, units), and minimum quantity threshold.
- Products shared across household (not global catalog per user).

F-03 — Inventory Management

- Each inventory item: product reference, quantity, unit, expiry date (optional), added-by, added-at.

- Batch entries supported (e.g., 'bought 3 cans of tomatoes' creates 3 entries with same expiry).
- Consume action decrements quantity; if quantity = 0, item removed and product added to shopping list.
- History log of all additions and consumptions for audit.

F-04 — Shopping List

- Auto-populated when inventory item falls below minimum threshold.
- Members can manually add items with name, quantity, and optional notes.
- Real-time sync: if one member checks off an item while shopping, others see it immediately.
- After shopping, list items can be bulk-moved to inventory (with quantity and expiry confirmation).

F-06 — Receipt Scanning (Highest Impact Feature)

Why this matters

This feature reduces per-product friction from ~10 seconds to under 1 second. One photo scan replaces up to 30 individual product entries — the key to sustaining user engagement beyond week 2.

- User photographs receipt OR scans NF-e QR Code (Brazil standard).
- Backend calls Claude Vision API to parse items, quantities, and prices from photo.
- For NF-e QR Code, backend fetches XML from SEFAZ and parses structured product data directly.
- User reviews parsed list, confirms quantities and expiry dates, then bulk-registers.
- Unrecognized products prompt user to match with existing catalog or create new entry.

4. Technical Architecture

4.1 Technology Stack

Layer	Technology / Library
Mobile App	React Native (Expo managed workflow) + Redux Toolkit + RTK Query
UI Library	React Native Paper 5.x — Material Design 3 component library. Provides themed components (Button, Card, TextInput, FAB, Snackbar, BottomNavigation, DataTable) consistent with Material You guidelines.
Navigation	React Navigation v6 + React Native Paper theming integration
Camera / Barcode	Expo Camera + Expo Barcode Scanner (ML Kit)
Location	expo-location — foreground GPS when app is open; saved address stored in DB for offline delivery suggestions.
State Management	Redux Toolkit + Redux Persist (offline support)
API Client	RTK Query with JWT interceptor
Internationalisation	i18next + react-i18next + expo-localization. Languages: English (en) and Brazilian Portuguese (pt-BR).
Push Notifications	Expo Notifications + Firebase Cloud Messaging
Backend Framework	.NET Core 10 Web API (Controllers — ApiController pattern)
Orchestration (local dev)	.NET Aspire — AppHost orchestrates API + PostgreSQL + RabbitMQ. Built-in dashboard for traces, logs, metrics.
Architecture Pattern	Modular Monolith — self-contained modules, single deployable unit
Build Configuration	Directory.Build.props + Directory.Packages.props (Central Package Management) at solution root.
Static Analysis	SonarAnalyzer.CSharp, Roslynator.Analyzers, StyleCop.Analyzers — run as Roslyn analysers at build time. .editorconfig enforces coding standards.
Observability	OpenTelemetry (traces, metrics, logs) → Aspire Dashboard (local) / Azure Monitor + Application Insights (production).
Repository	Monorepo: stoq/ with server/ (.NET), mobile/ (Expo), landing/ (Next.js/Astro), api-contracts/, and .github/workflows/. Independent CI pipelines via GitHub Actions path filters.
CI/CD	GitHub Actions: backend.yml (build → analyse → test → OpenAPI → Docker → deploy), mobile.yml (typecheck → lint → test → EAS build), api-codegen.yml (TypeScript client regen).
Identity	Microsoft ASP.NET Core Identity + JWT Bearer + OAuth2 (Google, Facebook, Apple)
Database	PostgreSQL 16 via Entity Framework Core 10 (Npgsql)

Layer	Technology / Library
Message Bus	RabbitMQ + MassTransit 8.x (last full free version) for inter-module communication
Real-Time	SignalR (ASP.NET Core) for household sync
AI / Vision	Anthropic Claude API (claude-sonnet) for photo recognition and receipt parsing
Barcode Lookup	Open Food Facts API (free tier)
NF-e Parsing	SEFAZ QR Code endpoint + custom XML parser
File Storage	Azure Blob Storage (product images, receipt photos)
Push (Server)	Firebase Admin SDK (FCM) from .NET backend
Hosting	Azure App Service (backend) + Azure Database for PostgreSQL
CI/CD	GitHub Actions → Azure Container Registry → App Service

4.2 Backend Architecture — Modular Monolith

Pattern: Modular Monolith

A single deployable .NET solution divided into strongly-bounded modules. Each module owns its own Domain, Application, and Infrastructure layers. Modules never reference each other directly — all cross-module communication goes through RabbitMQ via MassTransit 8.x. The API project is a thin entry-point layer that sits outside the modules and wires everything together. All HTTP endpoints are implemented as ApiControllers ([ApiController] attribute) — no Minimal API.

4.2.1 Solution Structure

The .NET solution is organized as follows:

Project	Responsibility
src/API/Stoq.API	Single Web API project. ApiController-based controllers, DI composition root, SignalR hubs, middleware pipeline. References Application layer of each module (interfaces only).
src/Modules/Identity/Stoq.Identity.Domain	User aggregate, Household aggregate, value objects (Email, InviteCode, Role), domain events (UserRegistered, HouseholdCreated, MemberJoined).
src/Modules/Identity/Stoq.Identity.Application	Use cases: RegisterUser, LoginUser, CreateHousehold, JoinHousehold. Ports: ITokenService, IEmailService. DTOs, command/query objects.

Project	Responsibility
src/Modules/Identity/Stoq.Identity.Infrastructure	EF Core DbContext (IdentityDbContext), ASP.NET Identity configuration, OAuth2 adapters, JWT service implementation, email adapter.
src/Modules/Catalog/Stoq.Catalog.Domain	Product aggregate, value objects (Barcode, ProductUnit, MinQuantity), domain events (ProductCreated, ProductUpdated).
src/Modules/Catalog/Stoq.Catalog.Application	Use cases: CreateProduct, LookupBarcode, IdentifyProductFromPhoto. Ports: IBarcodeRepository, IVisionService.
src/Modules/Catalog/Stoq.Catalog.Infrastructure	EF Core DbContext, Open Food Facts HTTP client, Claude Vision API adapter, Azure Blob Storage adapter.
src/Modules/Inventory/Stoq.Inventory.Domain	InventoryItem aggregate, ExpiryBatch value object, domain events (ItemAdded, ItemConsumed, ItemExpired, StockLow).
src/Modules/Inventory/Stoq.Inventory.Application	Use cases: AddInventoryItem, ConsumeItem, GetExpiringItems, BatchRegisterFromReceipt. Ports: IInventoryRepository, INotificationPort.
src/Modules/Inventory/Stoq.Inventory.Infrastructure	EF Core DbContext, Hangfire job for expiry scanning, FCM push adapter, NF-e SEFAZ HTTP client.
src/Modules/Shopping/Stoq.Shopping.Domain	ShoppingList aggregate, ShoppingItem entity, domain events (ItemAddedToList, ItemCheckedOff, ListCleared).
src/Modules/Shopping/Stoq.Shopping.Application	Use cases: AddToShoppingList, CheckOffItem, GenerateListFromThresholds. Ports: IShoppingListRepository.
src/Modules/Shopping/Stoq.Shopping.Infrastructure	EF Core DbContext, MassTransit consumers (listens for StockLow from Inventory module).
src/Modules/Retailer/Stoq.Retailer.Domain	Retailer aggregate, RetailerMember entity, value objects (ApiKey, RetailerRole), domain events (RetailerCreated, HouseholdOptedIn, HouseholdOptedOut).
src/Modules/Retailer/Stoq.Retailer.Application	Use cases: RegisterRetailer, InviteRetailerMember, GetConsumptionAnalytics, GetExpiringItemsAggregated, ManageHouseholdConsent. Ports: IRetailerRepository, IAnalyticsPort.

Project	Responsibility
src/Modules/Retailer/Stoq.Retailer.Infrastructure	EF Core DbContext, API key hashing adapter, read-model consumers (subscribes to Inventory events to build aggregated analytics tables), consent management.
src/Shared/Stoq.Shared.Contracts	Shared integration event contracts (message schemas) used by MassTransit publishers and consumers across modules. No domain logic here.
src/Shared/Stoq.Shared.Kernel	Base classes: AggregateRoot, DomainEvent, ValueObject. Common utilities (Guard clauses, Result type). No external dependencies.

4.2.2 Module Internal Layer Rules

Layer	Rules & Responsibilities
Domain	Pure C# — no framework dependencies. Contains entities, value objects, domain services, and domain events. Domain events are published in-process via MediatR (INotificationHandler) within the same module.
Application	References Domain only. Defines use case handlers (IRequestHandler via MediatR), ports (interfaces), DTOs, and command/query objects. No EF Core, no HTTP clients here.
Infrastructure	References Application (to implement ports) and Domain. Contains EF Core DbContext, HTTP clients, MassTransit producers/consumers, Azure adapters, and external service implementations.

Key Rule — No Direct Module References

A module's projects must NEVER reference another module's projects. Example: Inventory.Application cannot import Catalog.Domain. Cross-module data needs are satisfied via: (a) MassTransit integration events for async flows, or (b) a query-only anti-corruption adapter in Infrastructure that reads from the other module's DB schema (read model only, no writes).

4.2.3 Inter-Module Communication — RabbitMQ + MassTransit 8

All asynchronous cross-module communication is handled by MassTransit 8.x (last fully open-source version) over RabbitMQ. Integration event contracts live in Stoq.Shared.Contracts so both publisher and consumer depend only on the shared schema, not on each other.

UserId in Every Integration Event

All integration events must carry an InitiatedByUserId (Guid) field — the UserId of the user whose action triggered the event. This allows every downstream module and the audit log to attribute any

side-effect (auto-added shopping item, expiry alert, inventory initialization) back to the originating user, even across module boundaries.

Publisher → Consumer	Event Contract (key fields)	Purpose
Inventory → Shopping	StockLowIntegrationEvent { HouseholdId, ProductId, CurrentQty, MinQty, InitiatedByUserId }	Triggered when an inventory item falls below MinQuantity. Shopping module consumer auto-adds the product to the household shopping list, logging InitiatedByUserId as the actor.
Inventory → Shopping	ItemExpiredIntegrationEvent { HouseholdId, ProductId, ExpiryDate, InitiatedByUserId }	Triggered when a batch is past expiry. Shopping module consumer flags the item for repurchase.
Identity → Inventory	HouseholdCreatedIntegrationEvent { HouseholdId, InitiatedByUserId }	Triggers initialization of an empty inventory for the new household. InitiatedByUserId = the Admin who created the household.
Identity → Shopping	HouseholdCreatedIntegrationEvent { HouseholdId, InitiatedByUserId }	Triggers creation of an empty shopping list for the new household.
Inventory → Notifications	ExpiryAlertIntegrationEvent { HouseholdId, ProductId, ExpiryDate, InitiatedByUserId }	Published by Hangfire job (InitiatedByUserId = system/scheduler sentinel UUID). Consumer sends FCM push to all household members.

MassTransit 8.x Configuration Note

Use MassTransit 8.x (e.g. 8.3.x) — the last major version before the commercial licensing changes introduced in v9. Configure RabbitMQ transport via AddMassTransit(x => { x.UsingRabbitMq(...) }). Use the Outbox pattern (AddEntityFrameworkOutbox) per module to guarantee at-least-once delivery even if RabbitMQ is temporarily unavailable.

4.2.4 Key Architectural Decisions

Decision	Rationale
Modular Monolith	Single deployable unit with strong module boundaries. Simpler than microservices operationally, but boundaries are enforced in code — easy to extract a module to a microservice later if needed.
CQRS + MediatR	Separates read (Query) and write (Command) operations within each module. Keeps handlers small, focused, and independently testable.
MassTransit Outbox	Each module's EF Core DbContext includes a MassTransit Outbox table. Ensures integration events are persisted in the same DB transaction as domain changes, preventing lost messages.
Shared Contracts only	Modules share only the Contracts and Kernel projects. This keeps coupling minimal and prevents domain logic leaking across module boundaries.

Decision	Rationale
SignalR at API layer	SignalR hubs live in the API project. They subscribe to integration events via MassTransit consumers and push updates to connected clients — decoupled from module internals.
EF Core per module	Each module has its own DbContext and its own PostgreSQL schema (e.g. identity, catalog, inventory, shopping). Migrations are managed per module.
JWT + Refresh Tokens	Short-lived access tokens (15 min) + long-lived refresh tokens (30 days) stored securely in device keychain (Expo SecureStore).

4.3 Database Schema (Core Tables)

Each module owns its own PostgreSQL schema. All tables include CreatedAt, CreatedBy, UpdatedAt, and UpdatedBy audit columns. CreatedBy and UpdatedBy store the UserId of the user responsible for the operation, enabling full audit trail logging.

Audit Column Convention

Every table inherits four audit columns from the base entity: CreatedAt (timestamptz), CreatedBy (UserId, uuid), UpdatedAt (timestamptz, nullable), UpdatedBy (UserId, uuid, nullable). These are set automatically by a SaveChanges interceptor in each module's DbContext — controllers and use cases never set them manually. CreatedBy captures the authenticated user from ICurrentUserService at the time of insert; UpdatedBy is updated on any subsequent write.

Table (schema.name)	Domain Columns | Audit Columns (all tables)
identity.Users	Id, Email, DisplayName, AvatarUrl, PreferredLanguage (en/pt-BR), SystemRole (Consumer/RetailerAdmin/RetailerViewer/RetailerApiUser/PlatformAdmin) | CreatedAt, CreatedBy, UpdatedAt, UpdatedBy
identity.Households	Id, Name, InviteCode, InviteCodeExpiresAt, DefaultDeliveryAddressId | CreatedAt, CreatedBy, UpdatedAt, UpdatedBy
identity.HouseholdMembers	HouseholdId, UserId, Role (Admin/Member/Viewer), JoinedAt | CreatedAt, CreatedBy, UpdatedAt, UpdatedBy
identity.Addresses	Id, HouseholdId, UserId (nullable — null = household default), Label (Home/Work/Other), Street, Number, Complement, Neighbourhood, City, State, PostalCode, Country, Latitude, Longitude, IsHouseholdDefault | CreatedAt, CreatedBy, UpdatedAt, UpdatedBy
identity.LocationSnapshots	Id, UserId, HouseholdId, Latitude, Longitude, AccuracyMetres, CapturedAt — ephemeral foreground GPS pings. Retained for 24h only, used for nearby-store suggestions.
catalog.Products	Id, HouseholdId, Name, Brand, Category, Barcode, ImageUrl, DefaultUnit, MinQuantity | CreatedAt, CreatedBy, UpdatedAt, UpdatedBy
inventory.InventoryItems	Id, HouseholdId, ProductId, Quantity, Unit, ExpiryDate, Notes | CreatedAt, CreatedBy, UpdatedAt, UpdatedBy

Table (schema.name)	Domain Columns	Audit Columns (all tables)
inventory.InventoryLogs	Id, HouseholdId, ProductId, Action (Add/Consume/Expire), Delta, InitiatedBy (UserId)	CreatedAt, CreatedBy, UpdatedAt, UpdatedBy
shopping.ShoppingListItems	Id, HouseholdId, ProductId, RequestedQuantity, IsChecked, CheckedBy, Source (Auto/Manual)	CreatedAt, CreatedBy, UpdatedAt, UpdatedBy
retailer.Retailers	Id, CompanyName, TaxId, ContactEmail, ApiKeyHash, IsActive	CreatedAt, CreatedBy, UpdatedAt, UpdatedBy
retailer.RetailerMembers	RetailerId, UserId, Role (Admin/Viewer/ApiUser)	CreatedAt, CreatedBy, UpdatedAt, UpdatedBy
retailer.HouseholdConsents	RetailerId, HouseholdId, ConsentGrantedAt, ConsentRevokedAt (nullable), GrantedBy (UserId)	CreatedAt, CreatedBy, UpdatedAt, UpdatedBy
retailer.ConsumptionReadModel	RetailerId, Barcode, ProductName, Category, TotalConsumed, AvgDaysBetweenPurchases, HouseholdsCount — anonymised aggregate, rebuilt from Inventory events via MassTransit consumer. No PII.	
masstransit.*	Outbox tables per module schema — managed automatically by MassTransit EF Core provider.	

4.4 Location Strategy

Hybrid Approach: Saved Address + Foreground GPS

Location is managed at two levels: (1) a saved address per household (set once, used for delivery suggestions even offline), with optional per-member overrides for personal delivery; and (2) foreground GPS via expo-location when the app is open, used to detect nearby stores in real time. No background tracking — privacy-first.

Aspect	Detail
Household default address	Set by any household Admin in Household Settings. Used as the delivery address for auto-generated shopping suggestions when no GPS is available. Stored in identity.Addresses with IsHouseholdDefault = true.
Per-member address override	Each member can save their own delivery address (e.g. workplace). Used when that member initiates a delivery request. Stored in identity.Addresses with UserId set and IsHouseholdDefault = false.
Foreground GPS (expo-location)	Requested when the app enters Shopping List or Scan screens. Used for 'nearby stores' and delivery ETA suggestions. Location snapped to identity.LocationSnapshots; retained 24h only. User must grant foreground permission explicitly.
Privacy & LGPD compliance	GPS is foreground-only — no background tracking. LocationSnapshots are purged after 24h by a Hangfire job. Users can opt out of GPS entirely in App Settings and rely solely on saved address. Consent is captured at first GPS request.

Aspect	Detail
Retailer use of location	Retailers never receive raw coordinates. Location data is used only to match households to store catchment areas (server-side polygon check). Retailers see only aggregated counts (e.g. '142 opted-in households within 5km of Store #12').

4.5 Retailer API — Roles & Scope

Retailer Access Model (v1.0 — Foundation Only)

v1.0 establishes the role structure, consent model, and data schema for the Retailer API. The actual analytics endpoints are stubbed/documented but not fully implemented — this avoids premature optimisation while ensuring the architecture can support them without breaking changes. Full implementation is planned for v2.0.

System Roles (identity.Users.SystemRole)

All users have a SystemRole that controls their access tier. Household-level roles (Admin/Member/Viewer from HouseholdMembers) are orthogonal to SystemRole — a user can be a Consumer with Admin rights in their household, or a RetailerAdmin with no household at all.

SystemRole	Capabilities & Access
Consumer	Default role for all end-users. Access to household inventory, shopping list, and all consumer-facing features. Cannot access any Retailer API endpoints.
RetailerAdmin	Human administrator of a retail company account. Can register the company, invite RetailerViewers and RetailerApiUsers, manage consent requests, and view analytics dashboards. Authenticates via standard OAuth2/email login.
RetailerViewer	Read-only analytics access for a retail company. Can view consumption reports and expiry trend data for consented households. Cannot modify company settings or generate API keys. Authenticates via standard login.
RetailerApiUser	Machine/service account for automated retailer integrations. Authenticates via API Key (not OAuth2). API Key is hashed in retailer.RetailerMembers. Used for programmatic consumption of the Retailer API (e.g. feeding data into a retailer's own BI system).
PlatformAdmin	Anthropic/internal platform operator. Full access to all data and configuration. Reserved for support and operations — not exposed to external users.

Consent Model

- Households must explicitly opt in to share their data with a specific retailer. Consent is per-retailer — opting into Retailer A does not share data with Retailer B.
- Consent is granted by a household Admin from the Household Settings screen. A consent record is written to retailer.HouseholdConsents.

- Consent can be revoked at any time. On revocation, ConsentRevokedAt is set and the household's data is excluded from all future retailer queries. Historical aggregates are purged within 24h.
- Retailers cannot see which specific households consented — only aggregated, anonymised metrics. No PII is ever exposed through the Retailer API.

Retailer API Endpoints (stubbed in v1.0, full implementation in v2.0)

Endpoint	Access & Description
POST /api/retailer/register	RetailerAdmin: register a new retail company account.
POST /api/retailer/members	RetailerAdmin: invite a new RetailerViewer or generate an API key for a RetailerApiUser.
GET /api/retailer/analytics/consumption	RetailerAdmin / RetailerViewer / RetailerApiUser: aggregated consumption data (product, category, frequency) across opted-in households. Anonymised.
GET /api/retailer/analytics/expiring	RetailerAdmin / RetailerViewer / RetailerApiUser: products nearing expiry across opted-in households — useful for timely delivery offers.
GET /api/retailer/analytics/location-reach	RetailerAdmin: count of opted-in households within a configurable radius of a store location (polygon check, no raw coordinates returned).
GET /api/retailer/consents	RetailerAdmin: list of active/revoked consents (HouseholdId anonymised as opaque token, consent dates only).

4A. Frontend Architecture — React Native

4A.1 UI Library — React Native Paper (Material Design 3)

Decision: React Native Paper 5.x

React Native Paper is the most mature and actively maintained Material Design implementation for React Native. It supports Material Design 3 (Material You), is fully compatible with Expo, integrates with React Navigation theming, and supports dark mode out of the box. The main alternative (React Native Material You) is less stable and less actively maintained.

Aspect	Detail
Install	react-native-paper, react-native-safe-area-context, @expo/vector-icons (for MD3 icons via MaterialCommunityIcons)
Provider setup	Wrap the app root in <PaperProvider theme={theme}>. Theme defined in src/theme/index.ts using MD3LightTheme / MD3DarkTheme with custom brand colour tokens.
Key components used	Button, TextInput, Card, Chip, FAB, BottomNavigation, Appbar, Snackbar, Dialog, Portal, DataTable, ActivityIndicator, Badge, Divider, List.Item
Custom theme	Brand primary: #1E3A5F. Secondary/accent: #2E7D32. Define in src/theme/index.ts via MD3LightTheme.colors spread. All screens consume theme via useTheme() hook — no hardcoded colours in component files.
React Navigation integration	Use adaptNavigationTheme() from react-native-paper to synchronise Paper theme with React Navigation theme — consistent header, tab bar, and screen background colours.
Dark mode	Toggled via uiSlice (Redux). PaperProvider swaps between MD3LightTheme and MD3DarkTheme. User preference persisted in AsyncStorage.
Form inputs	All text inputs use Paper's TextInput (mode='outlined'). No raw React Native TextInput in screens — ensures consistent MD3 styling and error states.

4A.2 Project Structure

The React Native (Expo) project follows a feature-based folder structure. Each feature mirrors a backend module, keeping concerns co-located and making it easy for developers to find all code related to a given domain area.

Path	Responsibility
src/i18n/	Internationalisation. i18next + react-i18next setup. Translation files: en.json and pt-BR.json (one file per feature namespace). Language detection from device locale, with manual override stored in AsyncStorage.
src/app/	Expo Router entry point (if using file-based routing) or React Navigation root. Declares AuthStack and AppStack (tab navigator).

Path	Responsibility
src/features/auth/	Login, Register, Social Auth screens. Auth slice (Redux). RTK Query auth endpoints. Expo SecureStore token persistence.
src/features/inventory/	Inventory dashboard, product detail, add-product screens. Inventory slice. RTK Query inventory endpoints.
src/features/catalog/	Barcode scanner screen, photo recognition flow, manual product form. Catalog RTK Query endpoints.
src/features/shopping/	Shopping list screen, check-off flow, manual add item. Shopping slice. SignalR real-time sync hook.
src/features/receipts/	Receipt camera screen, NF-e QR scan, parsed items confirmation screen. Receipt RTK Query endpoints.
src/features/household/	Household creation, invite code screen, member management, address management, retailer consent settings. Household slice.
src/features/location/	Address form screen, GPS permission request, nearby-store suggestions. Location slice (stores active GPS coords and saved addresses). Uses expo-location.
src/store/	Redux store configuration. Root reducer. Redux Persist config (persists inventory and shopping slices to AsyncStorage).
src/api/	RTK Query baseApi definition with JWT interceptor and token refresh logic. All API slices injected here.
src/hooks/	Custom hooks: useSignalR, useBarcode, useCamera, usePushNotifications, useLocation (foreground GPS with permission gate).
src/components/	Shared Paper-based UI components: AppButton, AppCard, AppTextInput (wrapping Paper primitives with project conventions), ProductCard, ExpiryBadge, EmptyState, OfflineBanner.
src/navigation/	AuthStack, AppTabs (Inventory, Shopping, Scan, Profile), modal stack definitions. Themed with adaptNavigationTheme().
src/theme/	Paper MD3 theme tokens, dark/light variants, typography scale, spacing constants.
src/utils/	Date helpers, barcode format validators, quantity formatters, offline queue utilities, coordinate distance helpers.

4A.3 State Management Architecture

Pattern: RTK Query for server state, Redux slices for client state

Server data (inventory, shopping list, household) lives in RTK Query cache — no manual loading state or useEffect fetching. Client-only state (selected filters, UI mode, offline queue) lives in Redux slices. Redux Persist serializes the shopping list and inventory cache to AsyncStorage for offline access.

State Layer	What lives here
RTK Query (server state)	Inventory list, Shopping list, Product catalog, Household data. Auto-refetches on focus/reconnect. Cache tags invalidated on mutations.
authSlice (Redux)	accessToken, refreshToken, userId, householdId, systemRole. Persisted via Redux Persist to Expo SecureStore.
locationSlice (Redux)	Current GPS coords (foreground), permission status, active saved address, nearby-store results. GPS coords not persisted — refreshed on app open.
inventorySlice (Redux)	Optimistic update queue for offline consume/add actions. Flushed to API on reconnect.
shoppingSlice (Redux)	Local check-off state during shopping trip (prevents flickering from SignalR updates mid-trip).
uiSlice (Redux)	Active filters, scan mode, modal state. Not persisted.

4A.4 Real-Time Sync (SignalR)

The useSignalR hook establishes a SignalR connection on app foreground and joins the user's household group. On receiving server-pushed events, it invalidates the relevant RTK Query cache tags, triggering automatic refetch.

Event	Frontend Reaction
InventoryUpdated	Invalidates RTK Query 'Inventory' tag → list refetches automatically.
ShoppingListUpdated	Invalidates 'ShoppingList' tag. During active shopping trip, deferred to avoid UI flicker.
MemberJoined	Invalidates 'Household' tag. Shows in-app toast notification.
ExpiryAlert	Received as FCM push (background) or SignalR event (foreground). Updates expiry badge on Inventory tab.

4A.5 Offline Strategy

Action	Offline Behavior
View Inventory	Redux Persist cache. Works fully offline with last-synced data.
View Shopping List	Redux Persist cache. Fully offline.
Check off shopping item	Optimistic update in shoppingSlice. Queued mutation synced on reconnect.
Consume inventory item	Optimistic update in inventorySlice offline queue. Synced on reconnect.
Add product / Scan barcode	Requires connectivity (barcode lookup, AI vision). Shows 'offline' banner and disables scan.

Action	Offline Behavior
Receipt scanning	Requires connectivity. Disabled offline with clear user feedback.

4A.6 Navigation Structure

Navigator	Screens
AuthStack	Splash → Login → Register → ForgotPassword → SocialAuthCallback
AppTabs	Bottom tab navigator: (1) Inventory, (2) Shopping List, (3) Scan [FAB-style center tab], (4) Profile
InventoryStack	InventoryDashboard → ProductDetail → AddProduct → ConsumeProduct
ShoppingStack	ShoppingList → AddShoppingItem → ReceiptScan → ReceiptConfirm
ProfileStack	Profile → HouseholdSettings → ManageMembers → AddressManager → RetailerConsents → Notifications → AppSettings (Language, Dark Mode)
Modals	ExpiryPicker, QuantityEditor, InviteCode, ScanResult — rendered over any tab via modal stack

4A.7 Internationalisation (i18n)

Supported Languages at Launch

English (en) and Brazilian Portuguese (pt-BR). Language is detected automatically from the device locale on first launch. The user can override the language manually in App Settings, and the preference is persisted in AsyncStorage and synced to identity.Users.PreferredLanguage so the server can also respond in the correct language (e.g. push notification text, error messages).

Library & Setup

- Library: i18next + react-i18next. Chosen for namespace support, interpolation, pluralisation rules, and React Native compatibility.
- Install: i18next, react-i18next, expo-localization (for device locale detection).
- Initialise i18next in src/i18n/index.ts before the Redux store and navigation mount. Use initReactI18next plugin.
- Language detection order: (1) value stored in AsyncStorage (user preference), (2) expo-localization device locale, (3) fallback to 'en'.
- All translation calls use the useTranslation hook: const { t } = useTranslation('inventory'). Never use raw strings in JSX.

File Organisation

Path	Contents
src/i18n/locales/en/	One JSON file per feature namespace: common.json, auth.json, inventory.json, shopping.json, catalog.json, receipts.json, household.json, notifications.json
src/i18n/locales/pt-BR/	Mirror structure of the en/ folder. All keys must be present in both languages — missing keys fall back to English and are flagged in CI.
src/i18n/index.ts	i18next initialisation. Imports all namespace files statically (no lazy loading on mobile — bundle size is negligible for two languages).
src/i18n/types.ts	TypeScript declaration merging for i18next — gives full type-safety on t() calls so missing or mistyped keys are caught at compile time.

Key i18n Rules

Rule	Detail
No hardcoded strings	Every user-visible string (labels, placeholders, errors, push notification templates) must go through t(). Enforced by ESLint rule (eslint-plugin-i18next).
Pluralisation	Use i18next plural suffixes (_one, _other) for quantity strings: e.g. '{{count}} item' / '{{count}} items'.
Date & number formatting	Use Intl.DateTimeFormat and Intl.NumberFormat with the active locale — never hardcode date or decimal formats.
RTL readiness	Although pt-BR and en are both LTR, use logical CSS properties (marginStart, marginEnd) to keep the codebase RTL-ready for future languages.
Server language header	RTK Query baseQuery automatically sends Accept-Language: {{locale}} on every request. The .NET API reads this header to localise validation errors and notification text.
Translation CI check	GitHub Actions runs a script that diffs en/ and pt-BR/ key sets and fails the build if any key is missing in either language.

Backend i18n Support

- The .NET API reads the Accept-Language request header (populated automatically by RTK Query) to determine response language.
- Validation error messages use IStringLocalizer<T> with resource files (Resources/en.resx and Resources/pt-BR.resx) per module.
- Push notification text is localised server-side: the user's PreferredLanguage from identity.Users selects the correct FCM message template.
- Error response DTOs include a messageKey field (e.g. 'inventory.item_not_found') alongside the localised message, so the frontend can optionally override with its own translation.

4B. Repository, Aspire & Build Infrastructure

4B.1 Monorepo Structure

One repo, four independent workspaces

The entire project lives in a single Git repository (stoq) split into four top-level folders: server/ (.NET solution), mobile/ (Expo), landing/ (Next.js/Astro), and api-contracts/. They share a repo for unified branch protection, PRs, and documentation, but have fully independent CI pipelines triggered by path filters. Neither workspace has a runtime dependency on the other — the contract between them is the OpenAPI spec published to api-contracts/.

Path	Contents & Purpose
stoq/server/	.NET solution root. Contains Stoq.sln, Stoq.AppHost, Stoq.ServiceDefaults, Stoq.Web (Blazor), all module projects, Directory.Build.props, Directory.Packages.props, .editorconfig, stylecop.json.
stoq/mobile/	Expo/React Native project root. Contains package.json, app.json, tsconfig.json, src/, .editorconfig, .eslintrc.js, .prettierrc.
stoq/landing/	Next.js or Astro static site. Company landing page and marketing content. Deployed independently to Azure Static Web Apps. No runtime dependency on server/ or mobile/.
stoq/api-contracts/	Auto-generated OpenAPI spec (openapi.json). Committed by server CI on every main branch build. Mobile CI watches this file to trigger TypeScript client regeneration via openapi-typescript.
stoq/.github/workflows/	server.yml (path filter: server/**), mobile.yml (path filter: mobile/**), landing.yml (path filter: landing/**), api-codegen.yml (path filter: api-contracts/openapi.json).
stoq/README.md	Root-level documentation: repo overview, local dev prerequisites, how to start each workspace, links to architecture docs.

4B.2 .NET Aspire Architecture

What Aspire provides for Stoq

.NET Aspire is the orchestration and observability layer for the backend. In local development, a single dotnet run from Stoq.AppHost starts the API, PostgreSQL, and RabbitMQ as a coordinated stack with service discovery pre-wired. The Aspire dashboard provides a live view of traces, logs, and metrics without any external tooling. In production, Aspire manifests are used to generate Azure Container Apps deployment descriptors.

Component	Role
Stoq.AppHost	Aspire orchestrator project. Defines the distributed application graph: API + PostgreSQL + RabbitMQ. Wires service discovery so the API

Component	Role
	receives connection strings automatically. Entry point for local dev (dotnet run --project server/Stoq.AppHost).
Stoq.ServiceDefaults	Shared Aspire defaults applied to all .NET projects via AddServiceDefaults() extension. Registers OpenTelemetry (traces, metrics, logs), health checks (/health, /alive), HttpClient resilience (Polly retry/circuit-breaker), and service discovery.
PostgreSQL (Aspire resource)	Declared in AppHost as builder.AddPostgres("postgres"). Five named databases declared: stoq-identity, stoq-catalog, stoq-inventory, stoq-shopping, stoq-retailer. Each Infrastructure project receives its connection string via Aspire service discovery — no hardcoded connection strings.
RabbitMQ (Aspire resource)	Declared as builder.AddRabbitMQ("rabbitmq"). API project references it via WithReference(rabbitmq). MassTransit picks up the connection string from Aspire service discovery automatically.
Aspire Dashboard (local)	Available at https://localhost:18888 when running AppHost. Shows live traces (request waterfall, DB queries, MassTransit message flows), structured logs with trace correlation, and real-time metrics (request rate, error rate, DB pool usage).
Production export	OTLP exporter configured to send to Azure Monitor (Application Insights) via Azure.Monitor.OpenTelemetry.AspNetCore. Same traces/metrics/logs visible in Azure Portal. Switch controlled by APPLICATIONINSIGHTS_CONNECTION_STRING environment variable.

4B.3 Centralised .NET Build Configuration

Directory.Build.props

Placed at server/ (solution root). Applies to every .csproj in the solution automatically — no per-project duplication.

Property	Value & Rationale
TargetFramework	net10.0
Nullable	enable — enforces nullable reference types across the entire solution.
ImplicitUsings	enable — adds common global usings (System, System.Collections.Generic, etc.).
TreatWarningsAsErrors	true in CI (set via environment variable DOTNET_WARNASERROR). Warnings are allowed locally for development velocity.
LangVersion	latest — always use the latest stable C# language version.
EnforceCodeStyleInBuild	true — Roslyn code style rules run at build time, not just in the IDE.

Property	Value & Rationale
EnableNETAnalyzers	true — activates Microsoft.CodeAnalysis.NetAnalyzers (CA rules) for all projects.
AnalysisLevel	latest-recommended — uses the latest recommended ruleset.
GenerateDocumentationFile	true for API-facing projects (API, Application layers). Feeds Swagger/Swashbuckle XML comments.

Directory.Packages.props

Enables Central Package Management (ManagePackageVersionsCentrally=true). All NuGet package versions declared here. Individual .csproj files use PackageReference without a Version attribute.

Package Group	Packages & Notes
Aspire packages	Aspire.Hosting.* (AppHost), Aspire.Npgsql.EntityFrameworkCore.PostgreSQL, Aspire.RabbitMQ.Client, Aspire.StackExchange.Redis (future).
EF Core / Database	Microsoft.EntityFrameworkCore (10.x), Npgsql.EntityFrameworkCore.PostgreSQL, Microsoft.EntityFrameworkCore.Tools.
Identity / Auth	Microsoft.AspNetCore.Identity.EntityFrameworkCore, Microsoft.AspNetCore.Authentication.JwtBearer.
MassTransit	MassTransit (8.x), MassTransit.RabbitMQ, MassTransit.EntityFrameworkCore.
MediatR / Validation	MediatR, FluentValidation, FluentValidation.DependencyInjectionExtensions.
OpenTelemetry	OpenTelemetry.Exporter.OpenTelemetryProtocol, OpenTelemetry.Instrumentation.AspNetCore, OpenTelemetry.Instrumentation.Http, OpenTelemetry.Instrumentation.EntityFrameworkCore, Npgsql.OpenTelemetry, Azure.Monitor.OpenTelemetry.AspNetCore.
Static Analysis	SonarAnalyzer.CSharp, Roslynator.Analyzers, StyleCop.Analyzers — added to Shared.Kernel or a dedicated Analysis props import file.
Testing	xunit, xunit.runner.visualstudio, Moq (or NSubstitute), FluentAssertions, Microsoft.AspNetCore.Mvc.Testing, Testcontainers.PostgreSql, Testcontainers.RabbitMq.

4B.4 Static Code Analysis & Coding Standards

.editorconfig — single source of truth for code style

A single .editorconfig at server/ root covers all C# files in the solution. Roslyn reads it at build time (EnforceCodeStyleInBuild=true), and IDEs (VS, Rider, VS Code) apply it live. Rules here take precedence over per-project settings.

Category	Rules & Settings
Indentation & whitespace	indent_style=space, indent_size=4, tab_width=4, end_of_line=lf, insert_final_newline=true, trim_trailing_whitespace=true.
Encoding	charset=utf-8-bom for .cs files (required for C# XML doc comments), utf-8 for all others.
C# naming rules	PascalCase: types, methods, properties, constants. _camelCase: private instance fields. camelCase: parameters and local variables. Enforced as IDE1006 / SA naming rules.
Using directives	dotnet_sort_system_directives_first=true. dotnet_separate_import_directive_groups=false. csharp_using_directive_placement=outside_namespace.
Null checking	dotnet_style_null_propagation=true. csharp_style_throw_expression=true. csharp_style_conditional_delegate_call=true.
Modern C# features	csharp_style_expression_bodied_methods=when_on_single_line. csharp_prefer_simple_using_statement=true. csharp_style_pattern_matching_over_is_with_cast_check=true.
Analyser severities	dotnet_diagnostic.CA2007.severity=error (ConfigureAwait). dotnet_diagnostic.CA1031.severity=warning (catch specific). dotnet_diagnostic.SA1600.severity=suggestion (member docs — warning in CI via TreatWarningsAsErrors).
StyleCop config (stylecop.json)	DocumentationRules: DocumentInternalElements=false (only public API needs XML docs). LayoutRules: NewlineAtEndOfFile=require.
SonarAnalyzer	Runs as a Roslyn analyser via NuGet — no SonarQube server required in CI. Catches common bugs (S1481 unused variables, S3776 complexity), security hotspots (S2068 hardcoded credentials), and code smells.
Roslynator	Additional refactoring and diagnostic rules. Configured to error on RCS1018 (add accessibility modifiers) and warn on others.

4B.5 OpenTelemetry Setup

All observability is configured in Stoq.ServiceDefaults and applied to every .NET project via the AddServiceDefaults() extension method. No per-project boilerplate needed.

Pillar	Detail
Distributed Tracing	Traces API requests end-to-end: HTTP in → controller → MediatR handler → EF Core query → MassTransit publish → consumer. Spans correlated by TraceId propagated via W3C TraceContext headers. Custom spans added for domain operations via StoqActivitySource in Shared.Kernel.
Metrics	ASP.NET Core metrics (request rate, latency histograms, error rate), HTTP client metrics, EF Core query counts, RabbitMQ message rates. Custom metrics: inventory_items_added_total, receipts_scanned_total, ai_vision_calls_total.

Pillar	Detail
Structured Logging	Microsoft.Extensions.Logging with OpenTelemetry log exporter. Logs include TraceId and SpanId for correlation. Log level per namespace configurable via appsettings.json. PII fields (email, address) masked via custom log enricher.
Local Dev (Aspire)	Aspire dashboard at :18888 collects OTLP data from all services automatically. No external infra needed. Trace waterfall, log search, and metric graphs available immediately.
Production (Azure Monitor)	Azure.Monitor.OpenTelemetry.AspNetCore package auto-configures exporter when APPLICATIONINSIGHTS_CONNECTION_STRING is set. Traces appear in Application Insights as end-to-end transactions. Metrics in Azure Monitor metrics explorer. Logs in Log Analytics via Log Analytics workspace linked to App Insights.
Alerting	Azure Monitor alerts on: p99 latency > 1s (API endpoints), error rate > 1% (5-min window), AI vision call failure rate > 10%, RabbitMQ queue depth > 1000 messages.

4B.6 GitHub Actions — CI/CD Pipelines

Path-filtered independent pipelines

Four workflow files in .github/workflows/. server.yml triggers only when server/** changes. mobile.yml triggers only when mobile/** changes. landing.yml triggers only when landing/** changes. api-codegen.yml triggers when api-contracts/openapi.json changes (committed by server CI). This ensures a React Native or landing page change never triggers a .NET build and vice versa.

server.yml — .NET Backend Pipeline

Stage	Detail
Trigger	push / pull_request to main or develop, paths: server/**
Step 1 — Restore & Build	dotnet restore && dotnet build --no-restore -c Release. TreatWarningsAsErrors=true enforced here via /p:TreatWarningsAsErrors=true.
Step 2 — Static Analysis	Roslyn analysers (SonarAnalyzer, StyleCop, Roslynator) run as part of the build step — no separate tool invocation needed. Build fails if any error-level rule is violated.
Step 3 — Unit Tests	dotnet test --no-build --filter Category=Unit. Coverage collected via Coverlet. Coverage threshold: 80% lines (build fails below threshold).
Step 4 — Integration Tests	dotnet test --filter Category=Integration. Uses Testcontainers to spin up real PostgreSQL and RabbitMQ containers for each test run. No mocks for infrastructure.
Step 5 — Generate OpenAPI spec	dotnet run --project src/API/Stoq.API -- --generate-openapi-spec. Outputs openapi.json to api-contracts/. Committed back to repo via a git commit step (only on main branch).

Stage	Detail
Step 6 — Docker build & push	docker build + push to Azure Container Registry. Image tagged with git SHA and latest. Only on main branch.
Step 7 — Deploy	az containerapp update to Azure Container Apps. Uses Aspire-generated manifest. Only on main branch after Docker push succeeds.

mobile.yml — React Native / Expo Pipeline

Stage	Detail
Trigger	push / pull_request to main or develop, paths: mobile/**
Step 1 — Install	cd mobile && npm ci (locked via package-lock.json for reproducible installs).
Step 2 — TypeScript check	npx tsc --noEmit. Fails on any type error, including i18next key type errors.
Step 3 — Lint	npx eslint src/ --max-warnings 0. Includes eslint-plugin-i18next (no hardcoded strings), @typescript-eslint (type-aware rules), react-native community config.
Step 4 — Unit Tests	npx jest --coverage --ci. Coverage threshold: 70% lines. Jest config uses jest-expo preset.
Step 5 — Expo export check	npx expo export --platform all --output-dir /tmp/expo-export. Validates the bundle builds without errors for both iOS and Android targets.
Step 6 — EAS Build (main only)	npx eas-cli build --platform all --profile preview --non-interactive. Builds on Expo's EAS infrastructure. Produces .apk (Android) and .ipa (iOS) for internal distribution.

landing.yml — Landing Page Pipeline

Stage	Detail
Trigger	push / pull_request to main or develop, paths: landing/**
Step 1 — Install	cd landing && npm ci
Step 2 — Type check & Lint	npx tsc --noEmit && npx eslint src/ --max-warnings 0.
Step 3 — Build	npm run build. Validates static output compiles without errors (Next.js or Astro build).
Step 4 — Deploy (main only)	az staticwebapp deploy to Azure Static Web Apps. Automatic preview URLs generated for pull requests via Azure Static Web Apps staging environments.

api-codegen.yml — OpenAPI TypeScript Client Regeneration

Stage	Detail
Trigger	push to main, paths: api-contracts/openapi.json
Step 1 — Generate client	cd mobile && npx openapi-typescript ../api-contracts/openapi.json --output src/api/generated/schema.d.ts. Generates TypeScript types from OpenAPI spec.
Step 2 — Commit & PR	Opens an automated PR titled 'chore: update generated API types from openapi.json'. Requires review before merge — prevents breaking type changes from being silently applied.

5. Authentication Design

5.1 Identity Architecture

The backend uses ASP.NET Core Identity as the user store, extended with OAuth2 external login providers. The mobile app uses Expo AuthSession to handle OAuth redirects and receives a JWT access token + refresh token pair on successful login. The JWT payload includes the user's SystemRole claim, which controls access to consumer vs. retailer API surfaces.

5.2 Supported Login Methods

Provider	Implementation Notes
Email / Password	Standard ASP.NET Identity flow. Email confirmation required. Password reset via 6-digit OTP sent to email. Available to all SystemRoles.
Google	OAuth2 via Google Identity Platform. Requires google-services.json on Android, GoogleService-Info.plist on iOS. Consumer-facing only.
Facebook	OAuth2 via Facebook Login SDK. Requires Facebook App ID and App Secret in backend config. Consumer-facing only.
Apple ID	Sign in with Apple (required for iOS apps with social login). Handled via Expo AppleAuthentication. Consumer-facing only.
API Key	Machine authentication for RetailerApiUser accounts only. Key generated by RetailerAdmin, hashed and stored in retailer.RetailerMembers. Passed as Authorization: ApiKey {key} header. Not available to human users.

5.3 Token Flow

10. User authenticates (social, email, or API Key for machine accounts) on mobile app or via API.
11. App sends identity token / credentials to POST /api/auth/login.
12. Backend validates with provider (Google/Facebook/Apple APIs), local Identity store, or API Key hash.
13. Backend creates or retrieves ApplicationUser, generates JWT (15 min) containing sub, email, systemRole, and householdId claims. Also issues Refresh Token (30 days).
14. Tokens stored in device Secure Store (Expo SecureStore). systemRole claim read from JWT to control which screens and API calls are available.
15. RTK Query baseQuery intercepts 401 → calls POST /api/auth/refresh → retries original request.
16. Logout revokes refresh token server-side and clears device storage.

6. Step-by-Step Implementation Guide

6.1 Development Phases

Sprint	Deliverables	Duration	Team Focus
Phase 1	Project setup, repository, .NET solution structure, EF Core + PostgreSQL, CI/CD pipeline	1 week	Backend + DevOps
Phase 2	Authentication API (all 4 providers), JWT flow, Refresh Tokens, user + household endpoints	2 weeks	Backend
Phase 3	React Native app scaffold (Expo), Redux store, Auth screens (login, register, social), navigation shell	2 weeks	Frontend
Phase 4	Product Catalog API + Barcode lookup (Open Food Facts). Inventory CRUD API + EF Core models	1.5 weeks	Backend
Phase 5	Inventory screens: product list, add product (barcode + manual), inventory dashboard, consume flow	2 weeks	Frontend
Phase 6	Shopping List API + SignalR hub. Real-time sync between household members	1.5 weeks	Backend + Frontend
Phase 7	Expiry alerts: background job (Hangfire), push notifications (FCM), in-app expiry view	1 week	Backend + Frontend
Phase 8	Receipt scanning: Claude Vision API integration, NF-e QR Code parser, batch-register flow	2 weeks	Backend + Frontend
Phase 9	AI product photo recognition (Claude Vision), product image storage (Azure Blob)	1 week	Backend + Frontend
Phase 10	Polish, offline support (Redux Persist), E2E tests, TestFlight / Play Store internal testing	2 weeks	Full Team

6.2 Phase 1 — Project Setup

Monorepo Structure

17. Create the root repository folder: mkdir stoq && cd stoq && git init
18. Create top-level folders: mkdir server mobile landing api-contracts .github/workflows
19. Add root README.md and .gitignore (combine .NET and Node gitignore templates).

Server — Solution Scaffold

20. cd server && dotnet new sln -n Stoq
21. Create AppHost and ServiceDefaults: dotnet new aspire-apphost -n Stoq.AppHost && dotnet new aspire-servicedefaults -n Stoq.ServiceDefaults
22. Create shared projects: Stoq.Shared.Kernel and Stoq.Shared.Contracts
23. Create module projects for Identity, Catalog, Inventory, Shopping, Retailer — each with .Domain, .Application, and .Infrastructure class libraries.

24. Create Stoq.API Web API project and add reference to Stoq.ServiceDefaults.
25. Add all projects to Stoq.sln.

Server — Centralised Build Configuration

26. Create server/Directory.Build.props at solution root. Set shared properties for all projects: TargetFramework (net10.0), Nullable (enable), ImplicitUsings (enable), TreatWarningsAsErrors (true), LangVersion (latest), EnforceCodeStyleInBuild (true).
27. Create server/Directory.Packages.props at solution root. Enable ManagePackageVersionsCentrally=true. Declare all NuGet package versions here (MassTransit, MediatR, FluentValidation, EF Core, Npgsql, Aspire, OpenTelemetry, etc.) — individual .csproj files reference packages without version numbers.
28. Create server/.editorconfig at solution root. Enforce: indent_style=space, indent_size=4, end_of_line=lf, charset=utf-8-bom, C# naming conventions (PascalCase for types/methods, _camelCase for private fields), dotnet_sort_system_directives_first=true. Add Roslyn analyser severity rules: dotnet_analyzer_diagnostic.severity=warning as baseline.
29. Add static analysis packages to Directory.Packages.props: Microsoft.CodeAnalysis.NetAnalyzers (built-in, enabled via EnableNETAnalyzers=true in Directory.Build.props), SonarAnalyzer.CSharp, Roslynator.Analyzers, StyleCop.Analyzers. Add a stylecop.json config file at solution root to configure StyleCop rules.
30. Add a .globalconfig or AnalyzerConfig entries to elevate key rules to errors (e.g. CA2007 ConfigureAwait, CA1031 catch specific exceptions, SA1600 member documentation).

Server — Package Installation

31. Install core packages (versions declared in Directory.Packages.props): MassTransit 8.x + MassTransit.RabbitMQ + MassTransit.EntityFrameworkCore, MediatR, FluentValidation, Microsoft.AspNetCore.Identity.EntityFrameworkCore, Npgsql.EntityFrameworkCore.PostgreSQL, Microsoft.AspNetCore.Authentication.JwtBearer.
32. Install Aspire packages: Aspire.Hosting.RabbitMQ, Aspire.Hosting.PostgreSQL in AppHost. Install Aspire.RabbitMQ.Client and Aspire.Npgsql.EntityFrameworkCore.PostgreSQL in Infrastructure projects.
33. Install OpenTelemetry packages in Stoq.ServiceDefaults: OpenTelemetry.Exporter.OpenTelemetryProtocol, OpenTelemetry.Instrumentation.AspNetCore, OpenTelemetry.Instrumentation.Http, OpenTelemetry.Instrumentation.EntityFrameworkCore, OpenTelemetry.Instrumentation.MassTransit (community), Npgsql.OpenTelemetry.

Server — Aspire AppHost Configuration

34. In Stoq.AppHost/Program.cs: add PostgreSQL resource (builder.AddPostgres("postgres")), add RabbitMQ resource (builder.AddRabbitMQ("rabbitmq")), add Stoq.API project resource with references to both (WithReference(postgres).WithReference(rabbitmq)).
35. Configure per-module PostgreSQL databases as named databases on the shared PostgreSQL resource: .AddDatabase("stoq-identity"), .AddDatabase("stoq-catalog"), etc. Each module's Infrastructure project receives its connection string via Aspire service discovery.
36. Enable Aspire dashboard in local dev (default). Configure OTLP exporter to point to Aspire's built-in collector endpoint in development; point to Azure Monitor / Application Insights in production via environment variable.

Server — OpenTelemetry Setup

37. In Stoq.ServiceDefaults/Extensions.cs (the standard Aspire ServiceDefaults extension): call AddOpenTelemetry() with tracing (AddAspNetCoreInstrumentation, AddHttpClientInstrumentation, AddEntityFrameworkCoreInstrumentation, AddSource("MassTransit")), metrics (AddAspNetCoreInstrumentation, AddHttpClientInstrumentation, AddRuntimeInstrumentation), and logging (AddOpenTelemetryLogging with IncludeFormattedMessage=true).
38. Add custom ActivitySource in Stoq.Shared.Kernel (e.g. StoqActivitySource) for domain-level spans — use cases emit spans for key operations (AddInventoryItem, ParseReceipt, etc.).
39. Configure resource attributes: service.name per project, service.version from assembly version, deployment.environment from ASPNETCORE_ENVIRONMENT.
40. In production (Azure): export to Azure Monitor using Azure.Monitor.OpenTelemetry.AspNetCore package. In local dev: Aspire dashboard collects traces/metrics/logs automatically via OTLP.

Server — EF Core & Migrations

41. Configure one EF Core DbContext per module, each targeting its own PostgreSQL schema. Connection string injected via Aspire service discovery (builder.AddNpgsqlDbContext<IdentityDbContext>("stoq-identity")).
42. Run initial migrations per module: dotnet ef migrations add InitialCreate --project src/Modules/Identity/Stoq.Identity.Infrastructure --startup-project src/API/Stoq.API (repeat for each module).

Frontend (React Native + Expo)

43. cd ../mobile && npx create-expo-app . --template expo-template-blank-typescript
44. Install dependencies: @reduxjs/toolkit, react-redux, redux-persist, @react-navigation/native, react-native-paper, expo-camera, expo-barcode-scanner, expo-secure-store, expo-auth-session, expo-location, i18next, react-i18next, expo-localization, @microsoft/signalr
45. Set up React Native Paper: wrap app root in PaperProvider with custom MD3 theme. Connect to React Navigation via adaptNavigationTheme().
46. Set up i18n: initialise i18next with en and pt-BR namespaces. Configure expo-localization for device locale detection.
47. Configure Redux store with RTK Query API slice and Redux Persist for inventory and shopping list slices.
48. Set up ESLint with eslint-plugin-i18next, @typescript-eslint, and react-native community config. Add Prettier. Add .editorconfig at mobile root (indent_size=2, consistent with JS/TS convention).

6.3 Phase 2 — Authentication API

Endpoints to implement

Endpoint	Description
POST /api/auth/register	Email + password registration. Returns JWT + refresh token.
POST /api/auth/login	Email + password login. Returns JWT + refresh token.
POST /api/auth/social	Receives provider token (Google/Facebook/Apple). Validates externally, creates user if new.
POST /api/auth/refresh	Validates refresh token, returns new JWT pair.

Endpoint	Description
POST /api/auth/logout	Revokes refresh token server-side.
POST /api/auth/forgot-password	Sends OTP to email.
POST /api/auth/reset-password	Validates OTP and sets new password.

External Provider Validation

- Google: Validate id_token against https://oauth2.googleapis.com/tokeninfo
- Facebook: Validate access_token against https://graph.facebook.com/me
- Apple: Validate identity_token using Apple's public keys (JWKS endpoint)

6.4 Phase 4 — Product & Inventory API

Key Endpoints

Endpoint	Description
GET /api/products	List all household products (with filtering and search).
POST /api/products	Create product (manual or from barcode data).
GET /api/products/barcode/{code}	Lookup barcode in household catalog, then Open Food Facts.
POST /api/products/identify	Upload photo, returns AI-identified product details.
GET /api/inventory	Current household inventory with expiry grouping.
POST /api/inventory	Add inventory item(s) — supports batch.
PATCH /api/inventory/{id}/consume	Decrement quantity or mark as finished.
GET /api/inventory/expiring	Items expiring within N days.

6.5 Phase 6 — Shopping List + Real-Time Sync

SignalR Hub Design

HouseholdHub (/hubs/household)
Clients join a group by HouseholdId on connect. All inventory changes, shopping list updates, and member actions broadcast to the group. Frontend React Native app connects via @microsoft/signalr package.

- Hub methods: JoinHousehold(householdId), LeaveHousehold(householdId)
- Server events pushed to group: InventoryUpdated, ShoppingListUpdated, MemberJoined

- Frontend uses useSignalR custom hook; updates Redux store slices on event receipt.
- Offline: Redux Persist caches state. On reconnect, app calls GET /api/inventory and GET /api/shopping-list to reconcile.

6.6 Phase 8 — Receipt Scanning

NF-e QR Code Flow (Brazil)

49. User scans QR Code on fiscal receipt using Expo Barcode Scanner.
50. App sends QR Code URL to POST /api/receipts/parse-nfe.
51. Backend fetches NF-e XML from SEFAZ endpoint in the QR Code URL.
52. XML parser extracts: product name (xProd), quantity (qCom), unit (uCom), EAN barcode if available.
53. Items returned to app for user confirmation. User sets expiry dates and confirms.
54. POST /api/inventory/batch creates all items in a single transaction.

Photo Receipt Flow (fallback)

55. User photographs paper receipt.
56. App uploads image to POST /api/receipts/parse-photo.
57. Backend sends image to Claude Vision API with structured prompt requesting JSON list of items.
58. Claude returns item list; backend fuzzy-matches product names against household catalog.
59. User confirms matches and fills in any gaps (expiry dates, quantities).
60. Batch inventory update committed.

7. Non-Functional Requirements

Requirement	Specification
Performance	API p95 response time < 300ms for list endpoints; < 2s for AI vision endpoints.
Offline Support	Core features (view inventory, edit shopping list) work offline. Sync on reconnect.
Security	HTTPS everywhere. Tokens in Secure Store (not AsyncStorage). OWASP Mobile Top 10 reviewed.
Scalability	Stateless API; horizontal scaling via Azure App Service. Connection pooling via PgBouncer.
Availability	99.5% uptime target. Health check endpoint at /health for Azure probe.
Data Privacy	LGPD (Brazil) compliant. User data export and deletion on request. Receipt photos deleted after 24h.
Accessibility	WCAG AA for tap targets, font sizes, and color contrast. VoiceOver / TalkBack support.
App Size	Initial APK/IPA < 50MB. Receipt photos compressed to < 2MB before upload.

8. Out of Scope (v1.0)

- Fixed camera / smart home integration for automatic shelf monitoring.
- Price comparison between supermarkets.
- Automated delivery / cart integration with external retailers.
- Nutritional tracking or dietary planning features.
- Web dashboard (mobile-only for v1.0).
- Multi-household management from a single account.

Future Roadmap (v2.0)

Smart shopping suggestions based on purchase history, integration with iFood/Rappi for one-tap reorder, monetization via affiliate commissions on delivery orders, optional fixed-camera add-on for premium tier.

9. Risks & Mitigations

Risk	Description	Mitigation
User Abandonment	Friction in daily product registration is the #1 churn risk.	Receipt scan as primary entry method; 'quick consume' one-tap action; habit-forming notification design.
AI Cost	Claude Vision API calls add variable cost per receipt scan.	Cache parsed results; limit to 5 scans/day on free tier; receipt parsing charged only on premium.
NF-e Availability	SEFAZ endpoints are sometimes slow or unavailable.	Graceful fallback to photo scan; timeout at 5s; cache NF-e responses for 1h.
App Store Approval	Apple requires Sign in with Apple if any social login is offered.	Apple authentication is included from day 1. Reviewed Apple guidelines before design.
Barcode Coverage	Open Food Facts lacks Brazilian products.	Allow manual product creation; consider Cosmos (Brazilian barcode DB) as secondary source.